

WEB APPLICATION DEVELOPMENT

LAB 4 – HOMEWORK

EXERCISE 5: SEARCH FUNCTIONALITY

5.1: Create Search Form

```
<!-- Create Search Form -->

<form action="list_students.jsp" method="GET">
<input type="text" name="keyword" placeholder="Search by name or code...">
<button type="submit">Search</button>
<a href="list_students.jsp">Clear</a>
</form>
<br>
```

Search form in list_student.jsp

5.2: Implement Search Logic

```
//Get students
if (keyword != null && !keyword.trim().isEmpty()) {
    sql = "SELECT * FROM students WHERE full_name LIKE ? OR student_code LIKE ? OR major LIKE ? " +
        "ORDER BY " + sortBy + " " + order + " LIMIT ? OFFSET ?";
    pstmt = conn.prepareStatement(sql);
    pstmt.setString(1, "%" + keyword + "%");
    pstmt.setString(2, "%" + keyword + "%");
    pstmt.setString(3, "%" + keyword + "%");
    pstmt.setInt(4, recordsPerPage);
    pstmt.setInt(5, offset);
} else {
    sql = "SELECT * FROM students ORDER BY " + sortBy + " " + order + " LIMIT ? OFFSET ?";
    pstmt = conn.prepareStatement(sql);
    pstmt.setInt(1, recordsPerPage);
    pstmt.setInt(2, offset);
}
if (!hasResult) {
    %>
    <tr><td colspan="7" style="text-align:center;">No students found.</td></tr>
    <%
}
```

Print out message that no students found

Test cases:

Student Management System

+ Add New Student | Export to CSV

Search by name or code: [asldkfjasd] [Search] [Clear]

ID	Student Code	Full Name	Email	Major	Created At	Actions
No students found.						

Delete Selected

Search term with “asldkfjasd”

Student Management System

+ Add New Student | Export to CSV

Search by name or code: [John] [Search] [Clear]

ID	Student Code	Full Name	Email	Major	Created At	Actions	
☐	22	SV016	John Doe	huynhthuananh170705@gmail.com	Computer Science	2025-11-08 11:35:39.0	Edit Delete
☐	2	SV002	Emily Johnson	emily.j@email.com	Information Technology	2025-11-04 08:46:01.0	Edit Delete
☐	1	SV001	John Smith	john.smith@email.com	Computer Science	2025-11-04 08:46:01.0	Edit Delete

Delete Selected

Search term with “John”

Student Management System

+ Add New Student | Export to CSV

Search by name or code: [SV001] [Search] [Clear]

ID	Student Code	Full Name	Email	Major	Created At	Actions	
☐	1	SV001	John Smith	john.smith@email.com	Computer Science	2025-11-04 08:46:01.0	Edit Delete

Delete Selected

Search term with “SV001”



Student Management System

Add New Student

Export to CSV

Search by name or code

Search

Clear

	ID	Student Code	Full Name	Email	Major	Created At	Actions
☐	22	SV016	John Doe	huythuananh170705@gmail.com	Computer Science	2025-11-08 11:35:39.0	Edit Delete
☐	21	SV015	Lucas Tran	lucas.tran@email.com	Computer Science	2025-11-08 10:47:53.0	Edit Delete
☐	19	SV013	Benjamin Le	benjamin.le@email.com	Data Science	2025-11-08 10:47:53.0	Edit Delete
☐	17	SV011	William Hoang	william.h@email.com	Computer Science	2025-11-08 10:47:53.0	Edit Delete
☐	15	SV009	Daniel Kim	daniel.k@email.com	Data Science	2025-11-08 10:47:53.0	Edit Delete
☐	14	SV008	Sophia Tran	sophia.tran@email.com	Computer Science	2025-11-08 10:47:53.0	Edit Delete
☐	5	SV005	David Wilson	david.w@email.com	Computer Science	2025-11-04 08:46:01.0	Edit Delete
☐	4	SV004	Sarah Davis	sarah.d@email.com	Data Science	2025-11-04 08:46:01.0	Edit Delete
☐	1	SV001	John Smith	john.smith@email.com	Computer Science	2025-11-04 08:46:01.0	Edit Delete

0

Delete Selected

1

Search term with "Science"

Description about the flow of search form:

- Firstly, the user types a keyword to search for such as SV001 or Science and click Search button.
- After clicking the button, the form sends a GET requests to the list_students.jsp file.
- Secondly, in the list_students.jsp file, it receives the keyword that user typed via the function called `request.getParameter("keyword");` and assign it to variable called keyword.
- The Java code in this jsp file will check that there is any keyword that was typed. If yes, the searching is executing via SQL query. The query searches in database with full_name, student_code, or major columns.
- Finally, there is a while loop that print out each result with full columns and display them on the table.

EXERCISE 6: VALIDATION ENHANCEMENT

6.1: Email Validation

```
// Check email validation (only if email is not empty)
if (email != null && !email.trim().isEmpty()) {
    if (!email.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$")) {
        response.sendRedirect("add_student.jsp?error=Invalid email format");
        return;
    }
}
```

Check email validation code in process_add.jsp file

```
//Check email validation
if (!email.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$")) {
    response.sendRedirect("edit_student.jsp?id=" + idParam + "&error=Invalid email format");
    return;
}
```

Check email validation code in process_edit.jsp file

```
<form action="process_add.jsp" method="POST" onsubmit="return submitForm(this)">
  <div class="form-group">
    <label for="student_code">Student Code <span class="required">*</span></label>
    <input type="text" id="student_code" name="student_code"
      placeholder="e.g., SV001" required
      pattern="[A-Z]{2}[0-9]{3,}"
      title="Format: 2 uppercase Letters + 3+ digits">
  </div>
```

Action (direct file) and POST method in add_student.jsp file

```
<form action="process_edit.jsp" method="POST" onsubmit="return submitForm(this)">
  <input type="hidden" name="id" value="%= studentId %">
```

Action (direct file) and POST method in edit_student.jsp file

```
String studentCode = request.getParameter("student_code");
String fullName = request.getParameter("full_name");
String email = request.getParameter("email");
String major = request.getParameter("major");
```

Get data from the user's input

Description about the flow of email validation:

- After clicking the “Save Student” button, the browser sends POST request from add_student.jsp or edit_student.jsp, which sends data from the user’s input to process_add.jsp or process_edit file to check email validation.
- In the process_add.jsp or process_edit file, it receives the data and use it for checking validation by using if statements. If the input of email field is not empty, it continues to check the email format via regular expressions - regex. In my code, I updated the regex to make it can check the more complicated email input such as john@email. If the email is invalid, the process file will redirect to add_student.jsp or edit_student file with an error message about the invalid email format. If the email is valid, all data typed by user will be inserted to database via SQL statement and redirect to list_students.jsp file. Moreover, before checking email validation, if any fields of the form are empty, it also redirect to the form in add_student.jsp or edit_student.jsp file with an error message about required fields are missing.

Test cases:

 **Edit Student Information**

Student Code

SV017

Cannot be changed

Full Name *

Daniel Smith

Email

john@email.com

Major

Information Technology

 Update Cancel

 Student Management System							
 Student updated successfully							
 Add New Student  Export to CSV							
Search by name or code: Search Clear							
	ID	Student Code	Full Name	Email	Major	Created At	Actions
☐	23	SV017	Daniel Smith	john@email.com	Information Technology	2025-11-08 11:39:54.0	 Edit  Delete

Email input = john@email.com

 **Edit Student Information**

Student Code

SV017

Cannot be changed

Full Name *

Daniel Smith

Email

john.doe@company.co.uk

Major

Information Technology

 Update Cancel

Student Management System

Student updated successfully

Add New Student

Export to CSV

Search by name or code...

Search

Clear

ID	Student Code	Full Name	Email	Major	Created At	Actions
23	SV017	Daniel Smith	john.doe@company.co.uk	Information Technology	2025-11-08 11:39:54.0	Edit Delete

Email input = john.doe@company.co.uk

+ Add New Student

Student Code *

SV023

Full Name *

John Doe

Email

john@email

Major

Information Technology

Save Student

Cancel

+ Add New Student

Invalid email format

Student Code *

e.g., SV001

Full Name *

Enter full name

Email

student@email.com

Major

e.g., Computer Science

Save Student

Cancel

Email input = john@email

+ Add New Student

Student Code *

SV023

Full Name *

John Doe

Email

johnemail.com

Major

Information Technology

Save Student

Cancel

Email input = johnemail.com

6.2: Student Code Pattern Validation

```
// Check student code format
if (!studentCode.matches("[A-Z]{2}[0-9]{3,}$")) {
    response.sendRedirect("add_student.jsp?error=Invalid student code format (e.g. SV001, IT123)");
    return;
}
```

Check student code format validation in process_add.jsp file

```
<% if (request.getParameter("error") != null) { %>
    <div class="error">
        <%= request.getParameter("error") %>
    </div>
<% } %>
```

Display error message in add_student.jsp file

Description about the flow of student code pattern validation:

- After clicking the “Save Student” button, the browser sends POST request from add_student.jsp, which sends data from the user’s input to process_add.jsp file to check student code pattern validation.
- In the process_add.jsp file, it receives the data and use it for checking student code format by using if statements. If the input of student code is not matched with the given regex, it will be redirected to add_student.jsp file with an error message about the invalid student code format. Otherwise, all data typed by user will be inserted into the database via SQL statement and redirect to list_students.jsp file

Test cases: Examples: SV001, IT123, CS9999 (valid); sv001, S001, SV12 (invalid)

+ Add New Student

Student Code *

sv001

Full Name *

John Smith



Vui lòng khớp với định dạng được yêu cầu.
Format: 2 uppercase letters + 3+ digits

Student code = sv001

+ Add New Student

Student Code *

S001

Full Name *

John Smith



Vui lòng khớp với định dạng được yêu cầu.
Format: 2 uppercase letters + 3+ digits

Student code = S001

+ Add New Student

Student Code *

SV12

Full Name *

John Smith



Vui lòng khớp với định dạng được yêu cầu.
Format: 2 uppercase letters + 3+ digits

Student code = SV12

EXERCISE 7: USER EXPERIENCE IMPROVEMENTS

7.1: Pagination

Database Query with LIMIT:

```
//Get students
if (keyword != null && !keyword.trim().isEmpty()) {
    sql = "SELECT * FROM students WHERE full_name LIKE ? OR student_code LIKE ? OR major LIKE ? " +
        "ORDER BY " + sortBy + " " + order + " LIMIT ? OFFSET ?";
    pstmt = conn.prepareStatement(sql);
    pstmt.setString(1, "%" + keyword + "%");
    pstmt.setString(2, "%" + keyword + "%");
    pstmt.setString(3, "%" + keyword + "%");
    pstmt.setInt(4, recordsPerPage);
    pstmt.setInt(5, offset);
} else {
    sql = "SELECT * FROM students ORDER BY " + sortBy + " " + order + " LIMIT ? OFFSET ?";
    pstmt = conn.prepareStatement(sql);
    pstmt.setInt(1, recordsPerPage);
    pstmt.setInt(2, offset);
}
```

SQL statement with LIMIT ? OFFSET? to get students per page

Calculate Pagination:

```
String pageParam = request.getParameter("page");
try {
    if (pageParam != null && !pageParam.trim().isEmpty()) {
        currentPage = Integer.parseInt(pageParam);
        if (currentPage < 1) currentPage = 1;
    }
} catch (NumberFormatException e) {
    currentPage = 1;
}

//Pagination & search setup
int currentPage = 1;
int totalPages = 1;
int totalRecords = 0;
int recordsPerPage = 10;

if (keyword != null && !keyword.trim().isEmpty()) {
    queryString = "&keyword=" + keyword;
}

int offset = (currentPage - 1) * recordsPerPage;

//Count total records
if (keyword != null && !keyword.trim().isEmpty()) {
    sql = "SELECT COUNT(*) FROM students WHERE full_name LIKE ? OR student_code LIKE ? OR major LIKE ?";
    countStmt = conn.prepareStatement(sql);
    countStmt.setString(1, "%" + keyword + "%");
    countStmt.setString(2, "%" + keyword + "%");
    countStmt.setString(3, "%" + keyword + "%");
} else {
    countStmt = conn.prepareStatement("SELECT COUNT(*) FROM students");
}

countRs = countStmt.executeQuery();
if (countRs.next()) totalRecords = countRs.getInt(1);
totalPages = (int) Math.ceil((double) totalRecords / recordsPerPage);
```

Counting the number of records to determine the number of pages

Display Pagination Links:

```
<!-- Pagination -->
<div class="pagination">
    <% if (currentPage > 1) { %>
        <a href="List_students.jsp?page=<%= currentPage - 1 %>&sort=<%= sortBy %>&order=<%= order %>&queryString=<%= queryString %>">
            Previous</a>
    <% } %>

    <% for (int i = 1; i <= totalPages; i++) { %>
        <% if (i == currentPage) { %>
            <strong><%= i %></strong>
        <% } else { %>
            <a href="List_students.jsp?page=<%= i %>&sort=<%= sortBy %>&order=<%= order %>&queryString=<%= queryString %>"><%= i %>
            </a>
        <% } %>
    <% } %>

    <% if (currentPage < totalPages) { %>
        <a href="List_students.jsp?page=<%= currentPage + 1 %>&sort=<%= sortBy %>&order=<%= order %>&queryString=<%= queryString %>">
            Next</a>
    <% } %>
</div>
```

Description about the flow of pagination:

- The pagination divides the list of students into smaller groups of 10 records per page, which are easy to read for user.
- When user clicks a page link, the system sends request containing the page parameters. The list_students.jsp file reads this parameter, calculates the offset value based on the current page number, and executes the SQL query using LIMIT and OFFSET from the database.
- After that, the list_students.jsp file regenerates the pagination links at the bottom of the, highlighting the current page.

7.2: Improved UI/UX

a) Success/Error Message Styling

```
.message.success {
    background-color: #d4edda;
    color: #155724;
    border-left: 5px solid #28a745;
}
.message.error {
    background-color: #f8d7da;
    color: #721c24;
    border-left: 5px solid #dc3545;
}
```

Add distinct colors (green for success, red for error)

```
<% if (request.getParameter("message") != null) { %>    <script>
    <div class="message success">    <!-- Success/Error Message Styling -->
        <i>✓</i> <%= request.getParameter("message") %>    setTimeout(function() {
    </div>    }
    <% } %>    var messages = document.querySelectorAll('.message');
    messages.forEach(function(msg) {
    <% if (request.getParameter("error") != null) { %>        msg.style.transition = "opacity 0.5s ease";
    <div class="message error">        msg.style.opacity = '0';
        <i>X</i> <%= request.getParameter("error") %>        setTimeout(() => msg.style.display = 'none', 500);
    </div>    });
    <% } %>    }, 3000); // Auto-hide after 3 seconds
```

Add icons (✓ for success, X for error)

Auto-hide after 3 seconds

b) Loading States

```
<script>
    function submitForm(form) {
        var btn = form.querySelector('button[type="submit"]');
        if (btn) {
            btn.disabled = true;
            btn.textContent = 'Processing...';
        }
        return true;
    }
</script>
<form action="process_add.jsp" method="POST" onsubmit="return submitForm(this)">
```

Description of loading state: The loading state is implemented in the add_student.jsp or edit_student.jsp file processes to prevent duplicate form submissions. When the user clicks the **Save** or **Update** button in add_student.jsp or edit_student.jsp, a function is executed to disables the button and changes its text to “**Processing...**”. Therefore, user’s action is being handled and prevents accidental multiple clicks. While the button is disabled, the data form is sent to either process_add.jsp or process_edit.jsp, where the server validates the inputs, performs database operations, and then redirects the user back to list_students.jsp with a success or error message. When the new page is loaded, the process is completed, and the button becomes enable and available for the next form interaction. The loading state improves user experience and data consistency by ensuring that each submission is processed only once.

c) Responsive Table

```
/* Table scrollable on small screens and better mobile layout */
.table-responsive {
    overflow-x: auto;
}
@media (max-width: 768px) {
    table {
        font-size: 12px;
    }
    th, td {
        padding: 5px;
    }
}
<div class="table-responsive">
<table>
<thead>
```

BONUS EXERCISES

BONUS 1: Export to CSV

```
<a href="add_student.jsp" class="btn">+ Add New Student</a>
<a href="export_csv.jsp" class="btn">! Export to CSV</a>

response.setContentType("text/csv");
response.setHeader("Content-Disposition", "attachment; filename=\"students.csv\"");

// Connect to database
Connection conn = null;
Statement stmt = null;
ResultSet rs = null;

try {
    Class.forName("com.mysql.cj.jdbc.Driver");
    conn = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/student_management",
        "root",
        "123456"
    );

    // Loop through records and print CSV data
    while (rs.next()) {
        int id = rs.getInt("id");
        String code = rs.getString("student_code");
        String name = rs.getString("full_name");
        String email = rs.getString("email");
        String major = rs.getString("major");

        // Handle null values safely
        if (email == null) email = "";
        if (major == null) major = "";

        out.println(id + "," +
            "\"" + code + "\",\" " +
            "\"" + name.replace("\"", "\\\"") + "\"\" " +
            "\"" + email.replace("\"", "\\\"") + "\"\" " +
            "\"" + major.replace("\"", "\\\"") + "\"\"");
    }
}
```

Description: When the user clicks the “Export to CSV” button on list_students.jsp, the browser sends a request to export_csv.jsp. Firstly, it connects to the database and get all student records. After that, the export_csv.jsp file sets the HTTP response headers to indicate a file download by specifying the content type as "text/csv" and adding the Content-Disposition header, which prompts the browser to download the file instead of displaying it. The server then writes a CSV header row (e.g., “ID, Student Code, Full Name, Email, Major, Created At”) followed by one line for each student in the result set, separating values with commas. As the JSP loops through each record, it outputs the data directly to the response stream in CSV format. Finally, when all rows have been written, the browser automatically starts downloading the generated file named students.csv.

BONUS 2: Sort by Column

```
// Sorting setup
String sortBy = request.getParameter("sort");
String order = request.getParameter("order");

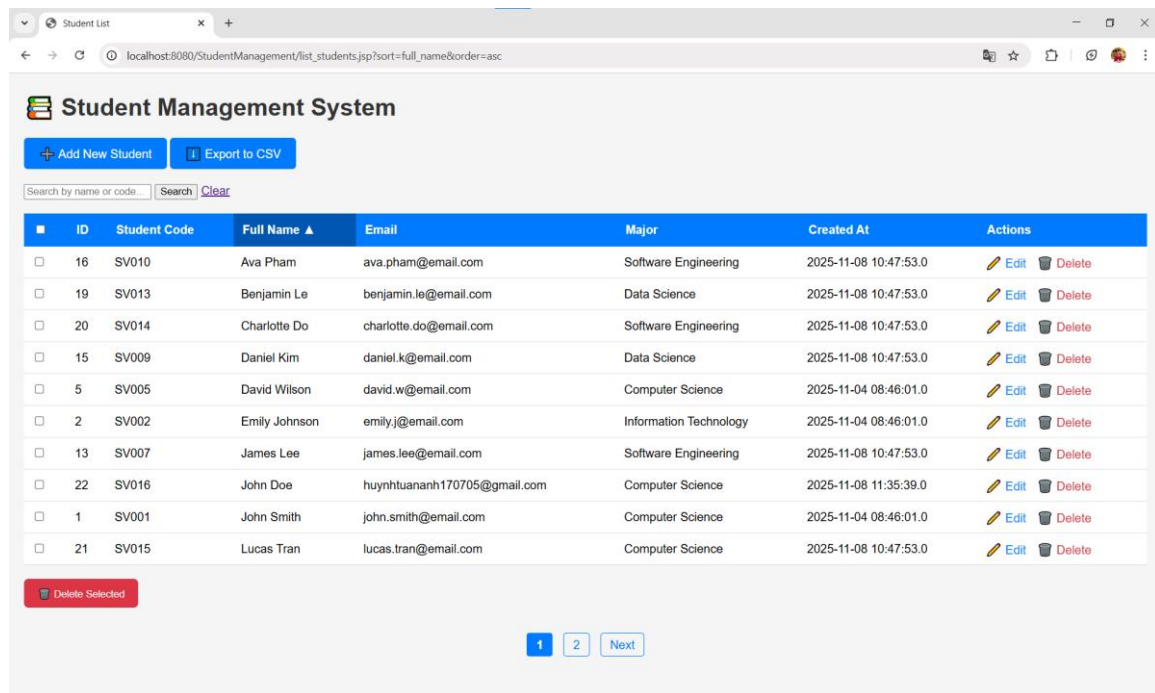
if (sortBy == null || sortBy.trim().isEmpty()) sortBy = "id";
if (order == null || order.trim().isEmpty()) order = "desc";

String nextOrder = order.equals("asc") ? "desc" : "asc";

<th>
    <a href="List_students.jsp?sort=full_name&order=<%= nextOrder %><%= queryString %>">Full Name</a>
    <% if ("full_name".equals(sortBy)) { %>
        <%= order.equals("asc") ? "▲" : "▼" %>
    <% } %>
</th>
<th>Email</th>
<th>Major</th>
<th>
    <a href="List_students.jsp?sort=created_at&order=<%= nextOrder %><%= queryString %>">Created At</a>
    <% if ("created_at".equals(sortBy)) { %>
        <%= order.equals("asc") ? "▲" : "▼" %>
    <% } %>
</th>
//Get students
if (keyword != null && !keyword.trim().isEmpty()) {
    sql = "SELECT * FROM students WHERE full_name LIKE ? OR student_code LIKE ? OR major LIKE ? " +
        "ORDER BY " + sortBy + " " + order + " LIMIT ? OFFSET ?";
    pstmt = conn.prepareStatement(sql);
    pstmt.setString(1, "%" + keyword + "%");
    pstmt.setString(2, "%" + keyword + "%");
    pstmt.setString(3, "%" + keyword + "%");
    pstmt.setInt(4, recordsPerPage);
    pstmt.setInt(5, offset);
} else {
    sql = "SELECT * FROM students ORDER BY " + sortBy + " " + order + " LIMIT ? OFFSET ?";
    pstmt = conn.prepareStatement(sql);
    pstmt.setInt(1, recordsPerPage);
    pstmt.setInt(2, offset);
}
```

Description: The sort-by-column feature allows users to organize student records which are sorted by Full Name or CreatedAt column. When the user clicks on one of these column headers in the student table, the system appends sorting parameters to the page URL - specifically sort (which indicates the column name) and order (which displays in ascending or descending). For example, clicking Full Name column will send a request to list_students.jsp file. Then, this file receives these parameters through request.getParameter("sort") and request.getParameter("order"), assigning default values (id and desc) if none are provided. After that, these values are worked directly with the SQL query's ORDER BY clause, ensuring that records are fetched from the database in the correct order - either alphabetically by name or chronologically by creation date. The page then returns with the sorted results, and the current sort of direction (▲ for ascending or ▼ for descending) is displayed beside the column title. Clicking the same header again toggles the order direction.

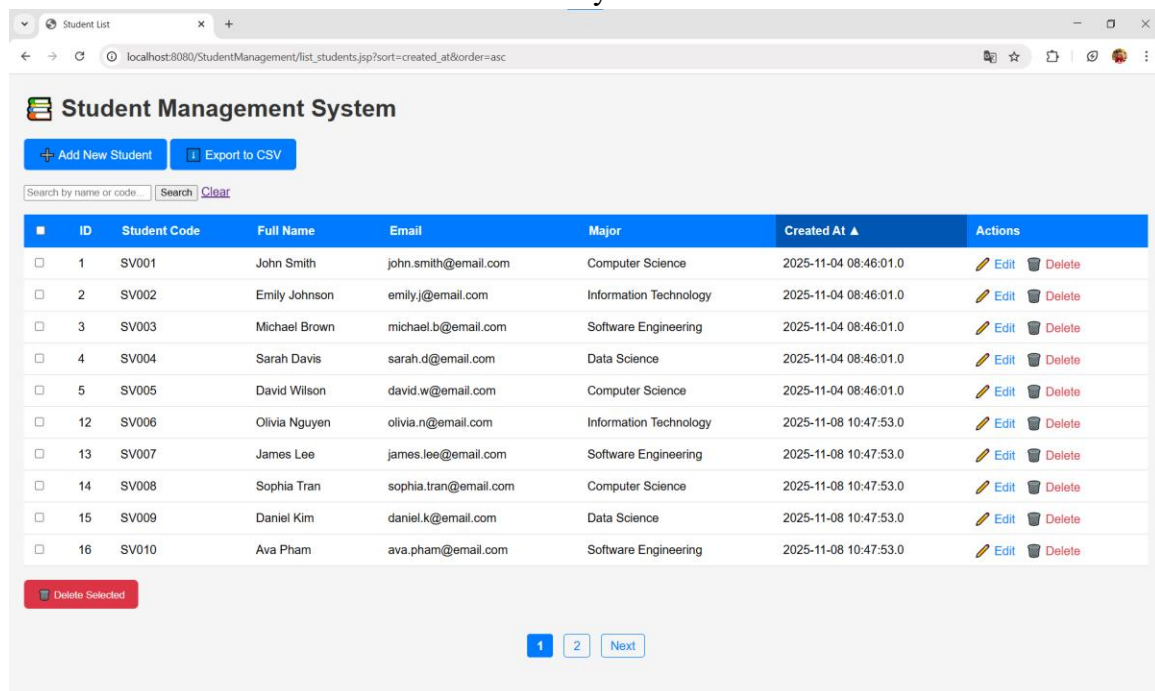
Test cases:



The screenshot shows the 'Student List' page of a 'Student Management System'. The URL is localhost:8080/StudentManagement/list_students.jsp?sort=full_name&order=asc. The table has 8 columns: ID, Student Code, Full Name ▲, Email, Major, Created At, and Actions. There are 11 student records displayed. The 'Full Name' column is sorted in ascending order. Below the table, there is a 'Delete Selected' button and pagination controls showing '1' of 2 pages with a 'Next' button.

ID	Student Code	Full Name ▲	Email	Major	Created At	Actions
16	SV010	Ava Pham	ava.pham@email.com	Software Engineering	2025-11-08 10:47:53.0	Edit Delete
19	SV013	Benjamin Le	benjamin.le@email.com	Data Science	2025-11-08 10:47:53.0	Edit Delete
20	SV014	Charlotte Do	charlotte.do@email.com	Software Engineering	2025-11-08 10:47:53.0	Edit Delete
15	SV009	Daniel Kim	daniel.k@email.com	Data Science	2025-11-08 10:47:53.0	Edit Delete
5	SV005	David Wilson	david.w@email.com	Computer Science	2025-11-04 08:46:01.0	Edit Delete
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	2025-11-04 08:46:01.0	Edit Delete
13	SV007	James Lee	james.lee@email.com	Software Engineering	2025-11-08 10:47:53.0	Edit Delete
22	SV016	John Doe	huynhtuananh170705@gmail.com	Computer Science	2025-11-08 11:35:39.0	Edit Delete
1	SV001	John Smith	john.smith@email.com	Computer Science	2025-11-04 08:46:01.0	Edit Delete
21	SV015	Lucas Tran	lucas.tran@email.com	Computer Science	2025-11-08 10:47:53.0	Edit Delete

Sorted by name



The screenshot shows the 'Student List' page of a 'Student Management System'. The URL is localhost:8080/StudentManagement/list_students.jsp?sort=created_at&order=asc. The table has 8 columns: ID, Student Code, Full Name, Email, Major, Created At ▲, and Actions. There are 16 student records displayed. The 'Created At' column is sorted in ascending order. Below the table, there is a 'Delete Selected' button and pagination controls showing '1' of 2 pages with a 'Next' button.

ID	Student Code	Full Name	Email	Major	Created At ▲	Actions
1	SV001	John Smith	john.smith@email.com	Computer Science	2025-11-04 08:46:01.0	Edit Delete
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	2025-11-04 08:46:01.0	Edit Delete
3	SV003	Michael Brown	michael.b@email.com	Software Engineering	2025-11-04 08:46:01.0	Edit Delete
4	SV004	Sarah Davis	sarah.d@email.com	Data Science	2025-11-04 08:46:01.0	Edit Delete
5	SV005	David Wilson	david.w@email.com	Computer Science	2025-11-04 08:46:01.0	Edit Delete
12	SV006	Olivia Nguyen	olivia.n@email.com	Information Technology	2025-11-08 10:47:53.0	Edit Delete
13	SV007	James Lee	james.lee@email.com	Software Engineering	2025-11-08 10:47:53.0	Edit Delete
14	SV008	Sophia Tran	sophia.tran@email.com	Computer Science	2025-11-08 10:47:53.0	Edit Delete
15	SV009	Daniel Kim	daniel.k@email.com	Data Science	2025-11-08 10:47:53.0	Edit Delete
16	SV010	Ava Pham	ava.pham@email.com	Software Engineering	2025-11-08 10:47:53.0	Edit Delete

Sorted by CreateAt

BONUS 3: Bulk Delete

```
//Select all and confirmation
function toggleSelectAll(source) {
    const checkboxes = document.querySelectorAll('input[name="selectedIds"]');
    checkboxes.forEach(cb => cb.checked = source.checked);
}

function confirmBulkDelete() {
    const selected = document.querySelectorAll('input[name="selectedIds"]:checked');
    if (selected.length === 0) {
        alert("Please select at least one student to delete.");
        return false;
    }
    return confirm("Are you sure you want to delete the selected students?");
}

<form action="bulk_delete.jsp" method="POST" onsubmit="return confirmBulkDelete();">
<div class="table-responsive">
<table>
    <thead>
        <tr>
            <th><input type="checkbox" id="selectAll" onclick="toggleSelectAll(this)"></th>
            <th>ID</th>
            <th>Student Code</th>
        </tr>
        <tr>
            <td><input type="checkbox" name="selectedIds" value="<%= id %>"></td>
            <td><%= id %></td>
            <td><%= studentCode %></td>
            <td><%= fullName %></td>
            <td><%= email != null ? email : "N/A" %></td>
            <td><%= major != null ? major : "N/A" %></td>
            <td><%= createdAt %></td>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td><input type="checkbox" name="selectedIds" value="<%= id %>"></td>
            <td><%= id %></td>
            <td><%= studentCode %></td>
            <td><%= fullName %></td>
            <td><%= email != null ? email : "N/A" %></td>
            <td><%= major != null ? major : "N/A" %></td>
            <td><%= createdAt %></td>
        </tr>
    </tbody>
</table>
</div>
</form>

String[] selectedIds = request.getParameterValues("selectedIds");

if (selectedIds == null || selectedIds.length == 0) {
    response.sendRedirect("list_students.jsp?error=No students selected for deletion");
    return;
}

try {
    Class.forName("com.mysql.cj.jdbc.Driver");
    conn = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/student_management",
        "root",
        "123456"
    );

    String sql = "DELETE FROM students WHERE id = ?";
    pstmt = conn.prepareStatement(sql);

    for (String id : selectedIds) {
        pstmt.setInt(1, Integer.parseInt(id));
        pstmt.addBatch();
    }

    int[] result = pstmt.executeBatch();

    response.sendRedirect("list_students.jsp?message=" + result.length + " students deleted successfully");
}
```

Description: The bulk delete feature allows users to remove multiple student records at once directly from the student list. The process begins when the user selects one or more checkboxes next to the student entries in the table and then clicks the “Delete Selected” button. These selected checkboxes each contain a unique student ID as their value. When the button is clicked, JavaScript first checks if any records are selected and displays a confirmation dialog to prevent accidental deletions. If confirmed, the form submits a POST request to `bulk_delete.jsp`, sending all selected IDs as an array (`selectedIds[]`). After that, at the server - side in the `bulk_delete.jsp` file receives this array using `request.getParameterValues("selectedIds")`, connects to the database, and executes a DELETE statement in a loop or batch process for each ID. After the deletion is completed, the page redirects back to `list_students.jsp` with a success message such as “3 students deleted successfully.” The updated student list then reloads, showing only the remaining records.